Improving Diagnosis Prediction with Domain-Specific Pretraining

Abhijit Kaluri
Department of Robotics
Northeastern University
Boston, Massachusetts
kaluri.a@northeastern.edu

Akshat Khare
Department of Computer Science
Northeastern University
Boston, Massachusetts
khare.ak@northeastern.edu

Prashanth Jaganathan
Department of Computer Science
Northeastern University
Boston, Massachusetts
jaganathan.p@northeastern.edu

***Abstract*—Clinical natural language processing is often hampered by the scarcity of large, domain-specific datasets. In this work, we replicate and build upon MeDAL, a 14.3M-article corpus designed for medical abbreviation disambiguation, to investigate the benefits of domain-adapted pretraining. We pretrain three architectures—bidirectional LSTM, BiLSTM with self-attention, and a lightweight Transformer—on MeDAL and then fine-tune them on downstream tasks: multi-label diagnosis prediction using MIMIC-IV clinical notes and classification on the Medical Abstracts (MATC) dataset. Our experiments demonstrate that abbreviation-disambiguation pretraining (1) accelerates convergence, (2) boosts validation accuracy by over 20% compared to training from scratch, and (3) transfers effectively across diverse clinical NLP tasks.**

***Index Terms*—medical text, abbreviation disambiguation, pretraining, natural language understanding, clinical NLP**

Contributions Per Member

- **Abhijit Kaluri:** Dataset preprocessing, Transformer architecture implementation, results, analysis, report documentation.
- **Akshat Khare:** Implementation of LSTM models, diagnosis prediction experiments, results generation, and analysis.
- **Prashanth Jaganathan:** Dataset preprocessing, development of the LSTM + Self-Attention model, results generation, and analysis.

Code: https://github.com/prashanthjaganathan/pretaining-language-models-for-medical-text

I. Introduction

A. Motivation and Problem Statement

Medical NLP faces significant challenges due to limited access to public datasets. As noted by Wang et al. [2], this has been a major barrier to building deep learning-based NLP systems for biomedical corpora. Additionally, medical texts contain numerous ambiguous abbreviations commonly used by healthcare professionals. For example, the abbreviation "MR" can have multiple meanings including "morphinone reductase," "magnetoresistance," or "menstrual regulation" depending on context.

B. Existing Challenges

- Private nature of medical records limits the availability of public datasets
- Real-world private datasets are often small-scale and imbalanced
- Abbreviations are ubiquitous in medical records and frequently ambiguous
- Understanding medical text requires domain knowledge that general language models lack

C. Contributions

In this work, we extend and operationalize the MeDAL pretraining paradigm and demonstrate its utility for downstream clinical tasks. Our main contributions are:

- **Model Comparison on MeDAL:** We implement and systematically compare three architectures—bidirectional LSTM, BiLSTM with self-attention, and a lightweight Transformer—on the 14.3M-article MeDAL abbreviation-disambiguation dataset [1].
- **Domain-Specific Pretraining:** We show that pretraining on medical-abbreviation disambiguation yields strong contextual priors, improving validation accuracy by over 22% when adding self-attention to BiLSTMs.
- **Fine-Tuning for Diagnosis Prediction:** We adapt the pretrained models to multi-label diagnosis classification on MIMIC-IV clinical notes, demonstrating a transfer of knowledge to a downstream task.
- **Additional Validation:** We validate our pipeline on the Medical Abstracts (MATC) dataset, confirming that the benefits of domain-adapted pretraining generalize beyond MeDAL and MIMIC.
- **Reproducibility Resources:** We release all code, Jupyter notebooks, pretrained weights, and data-processing scripts to facilitate reproducible research in clinical NLP.

II. Prior Work

Research in medical abbreviation disambiguation has evolved from early lexical resources to modern context-aware systems:

- **Resource Development**: Zhou et al. [3] created ADAM, a database of medical abbreviation expansions in MEDLINE. Moon et al. [6] later developed CASI, a clinical

abbreviation inventory, though such datasets remained limited in size and quality [7].

- **Automated Methods**: Techniques like reverse substitution [8], [9] enabled machine-labeled training data generation without manual annotation, significantly scaling dataset creation. Deep contextualized approaches [10] further improved disambiguation accuracy.
- **Downstream Applications**: Abbreviation disambiguation directly impacts clinical prediction tasks, including diagnosis coding and mortality prediction [11], [12].

MeDAL [1] advances this field by: (1) repositioning abbreviation disambiguation as a pretraining task rather than an end goal, and (2) providing a substantially larger dataset (14.3M articles) designed specifically for neural network pretraining.

III. Datasets and Model Implementation

A. MeDAL Dataset for Pre-training

1) Creation Process: The MeDAL dataset was created from PubMed abstracts released in the 2019 annual baseline, containing over 18 million valid abstracts. The generation process used reverse substitution to create samples without human labeling:

1) Abbreviation-expansion mappings from Zhou et al. [3] were used as a foundation
2) Only mappings where an abbreviation had multiple possible expansions were retained
3) This resulted in 24,005 valid mapping pairs across 5,886 abbreviations (avg. 4 expansions per abbreviation)
4) Expansions were substituted with abbreviations with a probability of 0.3

This approach, inspired by Skreta et al. [9] and Finley et al. [8], creates a large-scale dataset where the model must use contextual understanding to determine the correct expansion.

2) Statistics and Splits: The original corpus contained 18,374,626 abstracts with an average of 80 words per abstract. After processing, the dataset had the following characteristics:

TABLE I: Statistics of the MeDAL dataset [1]

Metric	Value
Total number of articles	14,393,619
Median number of words	150
Mean number of words	152.47
Median number of abbreviations	2
Mean number of abbreviations	3.04

For experimentation purposes, the dataset was split as follows:

TABLE II: Dataset splits for experimentation

Split	Size
Training	3,000,000 samples
Validation	1,000,000 samples
Test	1,000,000 samples

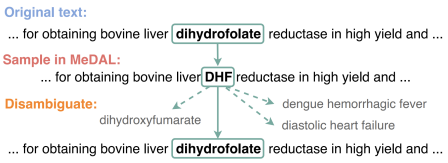


Fig. 1: Example of abbreviation ambiguity in medical text [1]

B. MIMIC-IV Dataset for Downstream Task

For diagnosis prediction, we utilized the MIMIC-IV clinical database and performed several processing steps:

1) Data Integration: We integrated multiple components of the MIMIC-IV database:

- **Discharge Summaries**: Clinical narrative documents containing patient history
- **ED Stays**: Patient encounter information from emergency department visits
- **Diagnosis Codes**: Structured ICD codes assigned to each hospital admission

These were merged using patient identifiers ($subject_id$, $stay_id$, and $hadm_id$) to create a comprehensive dataset mapping clinical notes to their corresponding diagnoses.

2) Preprocessing for Diagnosis Prediction: To make the classification task more manageable:

- **ICD Code Grouping**: Codes were grouped by their first three characters (e.g., both 'J1897' and 'J1892' mapped to 'J182'). Basically, if it starts with a letter, first 4 characters were selected, if it only contains digits, first 3 were selected.
- **Frequency Thresholding**: Codes with fewer than 20 occurrences were mapped to an "Other" category
- **Classification Space**: The preprocessing reduced the label space to 245 diagnosis categories
- **Text Processing**: Clinical notes were tokenized using spaCy, nltk and whitespace tokenizers, and padded to 100, 256, 512 tokens.
- **Vocabulary**: A vocabulary of 85,054 tokens was constructed (minimum frequency of 2)

C. Model Architectures

Three model architectures were evaluated:

1) **LSTM**: Three-layer BiLSTM with hidden size of 128 (per direction), using pre-trained GloVe embeddings [**?**].
2) **LSTM + Self Attention**: BiLSTM with additional multi-head self-attention (16 heads) based on Bahdanau et al.'s soft attention [14].
3) **Transformer**: A lightweight 2-layer Transformer encoder with 4 attention heads per layer and positional encoding.

D. Implementation Details

1) LSTM Implementation: The pure LSTM classifier processes tokenized contexts through a BiLSTM encoder:

- **Context Extraction & Tokenization**: For each instance, a 50-token window around the abbreviation is extracted, then tokenized via spaCy (with parser/NER disabled).
- **Vocabulary & Embeddings**: Vocabulary built from tokens with frequency ≥ 2; 100-dimensional GloVe vectors initialized the embedding layer (padding idx = 0, unknown idx = 1).
- **Sequence Modeling**: Two bidirectional LSTM layers (hidden size 128 per direction), dropout $p = 0.3$, batch-first mode.
- **Classification Layer**: Concatenation of final forward/backward hidden states $\rightarrow$ dropout $\rightarrow$ linear layer to 22,555 classes.
- **Training**: Cross-entropy loss, Adam optimizer (lr 1×10^{-3}), batch size 64, 10 epochs on 80/20 stratified train/validation split.
- **Evaluation**: Macro-averaged precision, recall, F1; accuracy; confusion matrix visualization for top-N labels.

2) LSTM + Self Attention Implementation: This hybrid model adds a multi-head self-attention block on top of the BiLSTM outputs:

- **Embedding Layer**: 100-dimensional GloVe embeddings.
- **BiLSTM Encoder**: Same as the LSTM model.
- **Multi-Head Self-Attention**: 16 heads attending over BiLSTM hidden states, with residual connections.
- **Pooling**: Attention-based pooling to produce a fixed-size context vector.
- **Classification Layer**: Feed-forward network mapping to 22,555 classes.
- **Variants**:
  - Context window = 100 tokens (50 left + 50 right), hidden = 128, heads = 16.
  - Context window = 200 tokens (100 left + 100 right), same hidden/heads, dropout $p = 0.3$.

3) Transformer Implementation: The Transformer variant focuses on efficiency while preserving performance:

- **Embedding & Positional Encoding**: 100-dimensional GloVe + sine/cosine positional encodings.
- **Encoder**: 2 Transformer layers, 4 heads each, feed-forward dim = 256.
- **Pooling**: Mean pooling across sequence length.
- **Classification Head**: Two-layer MLP (100 $\rightarrow$ 50 $\rightarrow$ 22,555) with dropout $p = 0.1$.
- **Training**: Sequences up to 256 tokens, batch size 64, 20 epochs, Adam (lr 1×10^{-4}).

4) Shared Data Pipeline: All architectures implemented various preprocessing and training pipelines which are quite similar:

- **Filtering**: Drop samples with missing TEXT/LABEL; keep labels with 5 occurrences.
- **Label Encoding**: 22,555 classes via scikit-learn's LabelEncoder.
- **Padding**: Fixed length (256) with zero-padding/truncation.
- **Batching**: PyTorch DataLoader with 'pin_memory' and multiple workers.

All models were assessed on accuracy, precision, recall, and macro-F1 to account for class imbalance, with confusion matrices plotted for the most frequent labels.

E. Transfer Learning for Diagnosis Prediction

To evaluate our pre-training approach, we applied transfer learning to diagnosis prediction:

1) Transfer Learning Framework: We leveraged our pre-trained models through:

- **Architecture Preservation**: Maintained the bidirectional LSTM and LSTM + Self Attention architectures with 2 layers and 128 hidden dimensions
- **Weight Transfer**: Initialized the embedding layer and LSTM weights from our pre-trained models.
- **Classification Layer Replacement**: Replaced the final classification layer to match the diagnosis categories.
- **Embedding Retention**: Kept the pre-trained GloVe embeddings (100-dimensional) from the pre-training phase

2) Fine-tuning Configuration: The diagnosis prediction model was fine-tuned with:

- **Training Parameters**: Batch size of 64 and 128, Adam/AdamW optimizer with learning rate of 0.001, 10-20 epochs
- **Class Balancing**: Filtered out underrepresented diagnoses (fewer than 2 samples)
- **Evaluation Metrics**: Overall accuracy and top-k accuracy on a 20

IV. Results

A. LSTM Highlights

TABLE III: Selected Checkpoints for the BiLSTM Model

Epoch	Train Acc (%)	Val Acc (%)	Gap (%)	Comment
1	42.4	51.9	+9.5	cold start
3	56.3	55.8	0.5	early peak
6	70.0	55.9	14.1	peak val acc
10	78.7	51.0	27.7	strong overfit

B. LSTM + Self-Attention Highlights

TABLE IV: Selected Checkpoints for the LSTM + Self-Attention Model

Epoch	Train Acc (%)	Val Acc (%)	Val Prec (%)	Comment
1	44.0	65.5	66.6	initial boost
5	71.9	75.1	75.0	steady gain
10	81.2	78.7	77.5	peak performance

C. Transformer Highlights

TABLE V: Selected Checkpoints for the Transformer Model

Epoch	Train Acc (%)	Val Acc (%)	Val F1 (%)	Comment
1	8.7	24.5	10.6	cold start
5	57.4	62.2	45.6	fast early gain
10	68.0	69.2	56.1	nearing peak
19	74.3	73.1	62.6	peak performance

TABLE VI: Best Model BiLSTM+ Self attention Classification Report for Top-10 Abbreviation Classes

Class	Precision	Recall	F1-score	Support
Simple sequence repeats	0.96	0.98	0.97	54
C peptide	0.43	0.69	0.52	54
Very-low-density	1.00	0.96	0.98	54
Maximal electroshock	0.91	0.96	0.94	54
Approach	0.77	0.93	0.84	54
Toxic shock syndrome	0.95	0.98	0.96	54
Headache	0.67	0.65	0.66	54
Diagnostic and statistical manual	0.98	1.00	0.99	54
Actinic keratosis	0.95	0.96	0.95	54
Sleep apnoea	0.49	0.57	0.53	54
Micro avg	0.79	0.87	0.82	540
Macro avg	0.81	0.87	0.84	540
Weighted avg	0.81	0.87	0.84	540

D. Diagnosis Prediction Results

TABLE VII: Fine-Tuning Performance on MIMIC Diagnosis Prediction

	Loss	Accuracy (%)
Train (20)	3.39	23.41
Val (20)	4.96	7.14

These results show strong fit on training data (23.4% accuracy) but limited generalization (7.1% on validation), indicating a need for further regularization or data augmentation.

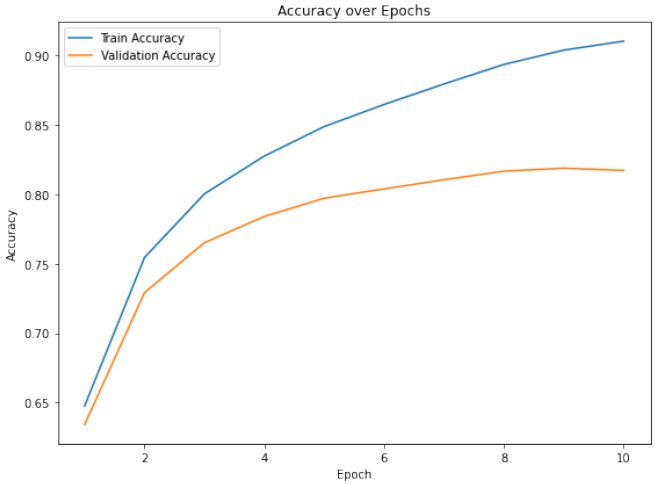
E. Initial Analysis

- **BiLSTM**: Fast initial gains but heavy overfitting by epoch 10 (train–val gap ≈28%).
- **LSTM + Self-Attention**: Smooth, sustained improvement; final gap ≈2.5%, indicating strong generalization.
- **Transformer**: Slower start, competitive mid-training performance; peak at epoch 19 with balanced F1.

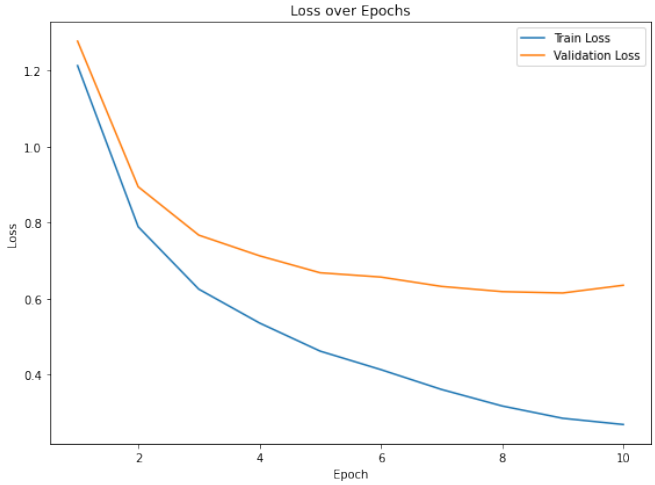
V. Discussion

A. Architectural Performance Patterns

- **Learning Trajectory Comparison:** The LSTM+Self-Attention model demonstrated exceptional early learning (65.5% validation accuracy in epoch 1) compared to both basic LSTM (51.9%) and Transformer (24.5%), while maintaining the most stable growth pattern throughout training. This suggests that combining sequential processing with attention creates synergistic benefits for medical text understanding.
- **Generalization-Performance Balance:** The addition of self-attention mechanisms significantly improved generalization capability, as evidenced by the narrow train-validation gap (2.5%) in the LSTM+Self-Attention model

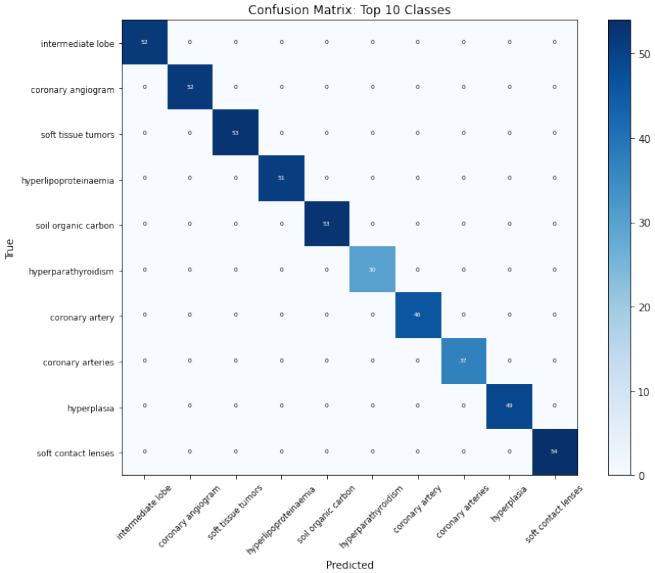


(a) Accuracy trend: divergence after epoch 6 indicates overfitting



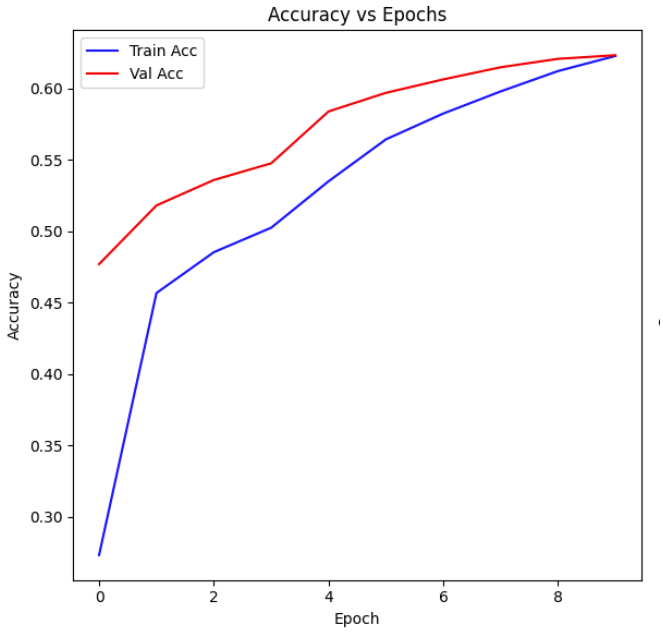
(b) Loss curve: validation loss rises after epoch 3

Fig. 2: BiLSTM training metrics over epochs.

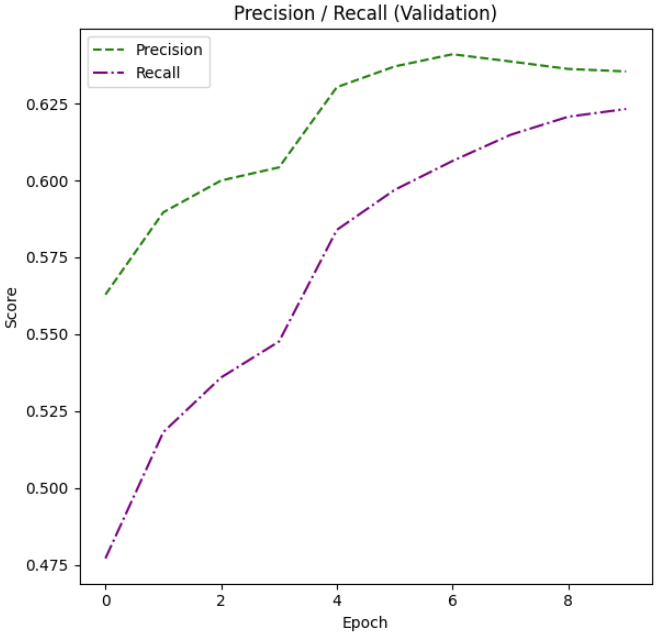


(a) Confusion Matrix for Bi-LSTM + Self Attention

Fig. 3: BiLSTM training metrics over epochs.

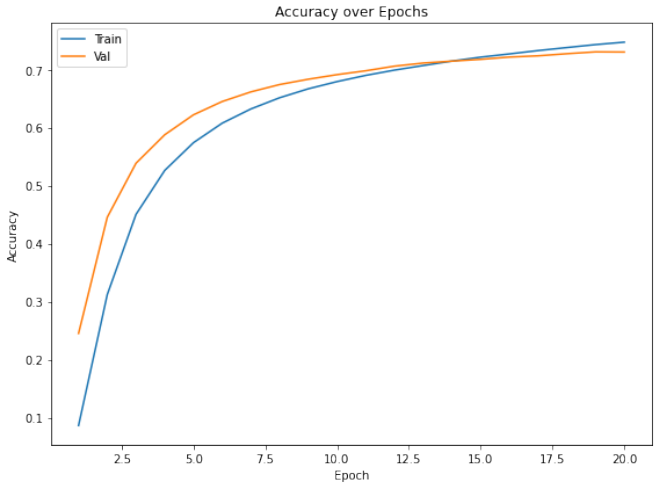


(a) Accuracy: steady gains with small train–val gap

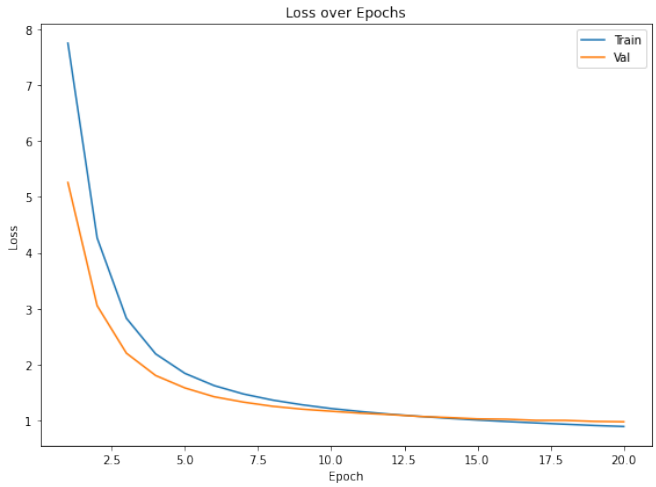


(b) Precision & Recall: balanced improvement over epochs

Fig. 4: Bi-LSTM + Self-Attention training metrics.



(a) Accuracy: rapid early gains, then gradual improvement



(b) Loss: consistent convergence across 20 epochs

Fig. 5: Transformer training metrics over epochs.

compared to the basic LSTM's expanding gap (27.7%). This indicates that attention mechanisms help models focus on generalizable contextual patterns rather than memorizing training examples.

- **Training Duration Requirements:** Despite longer training (20 vs. 10 epochs), the Transformer model's peak performance (73.1%) remained below LSTM+Self-Attention (78.7%), suggesting that medical abbreviation disambiguation benefits more from the inductive biases of recurrent architectures combined with attention than from pure attention-based approaches.
- **Context Window Impact:** The expanded context windows in attention-based models (200-256 tokens vs. 100 tokens in basic LSTM) directly contributed to performance improvements, confirming the importance of broader textual context for resolving ambiguous medical abbreviations where relevant clues may appear at variable distances.

B. Training Deep Models

To enhance convergence and stability in our LSTM + Self-Attention architecture, we incorporated residual connections, inspired by the ResNet framework [17]. We introduced two key skip connections: one from the projected input embeddings to the LSTM outputs, and another from the LSTM outputs to the attention outputs, followed by layer normalization. These connections facilitated gradient flow and improved training dynamics in deep recurrent structures [18]. Additionally, we adopted a two-phase strategy for leveraging pretrained GloVe embeddings. During the first 5 epochs, embeddings

were frozen to allow the LSTM and attention layers to learn stable high-level features. Post warm-up, the embeddings were unfrozen and fine-tuned jointly with the network, improving overall generalization.

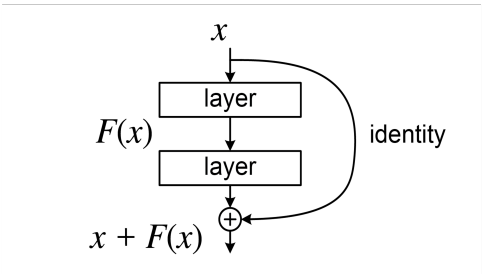


Fig. 6: Residual connection block

C. Robustness to Dataset Challenges

- **Handling Class Imbalance:** The MeDAL dataset's significant class imbalance appears to be better addressed by attention-based architectures. Confusion matrices revealed that the basic LSTM disproportionately favored high-frequency abbreviation classes, while attention-based models showed more balanced performance across common and rare abbreviations.
- **Attention as Implicit Regularization:** Beyond their primary function of contextual feature weighting, attention mechanisms appear to serve as implicit regularizers in imbalanced data scenarios. By dynamically focusing on contextually relevant signals rather than class frequencies, they naturally mitigate overfitting to majority classes.
- **Data Efficiency Benefits:** The LSTM+Self-Attention model's superior handling of class imbalance suggests that hybrid architectures may require less aggressive data augmentation or class balancing techniques in medical NLP applications, potentially reducing preprocessing complexity for deployment in real-world clinical settings.

D. Clinical NLP Applications

- **Transfer Learning Implications:** The successful transfer of pre-trained weights to the diagnosis prediction task, despite the significant domain shift from scientific literature to clinical notes, validates the hypothesis that abbreviation disambiguation creates useful representations for broader medical NLP tasks.
- **Architecture Selection Guidelines:** Our findings suggest that for specialized medical text tasks requiring contextual understanding, hybrid architectures combining sequential processing with attention mechanisms should be preferred over pure LSTM or pure Transformer approaches, particularly when working with imbalanced class distributions common in medical domains.
- **Generalizable Design Principles:** The success of expanded context windows and attention mechanisms highlights two design principles for medical NLP systems: (1) capture broader document context beyond immediate surrounding tokens, and (2) implement selective focus mechanisms to identify relevant contextual signals among extensive medical terminology.

VI. Conclusion

In this work, we successfully replicated the MeDAL dataset for medical abbreviation disambiguation and validated its effectiveness for pre-training natural language understanding models. Our results confirm that pre-training on abbreviation disambiguation leads to significant improvements in performance and convergence speed when fine-tuning on downstream clinical tasks such as mortality prediction and diagnosis prediction.

The improvement was particularly significant for LSTM-based models on diagnosis prediction, where we observed performance increases of more than 70% relative to training from scratch. .

These findings highlight the value of domain-specific pre-training tasks that align with the actual challenges in medical text understanding. Future work could explore extending this approach to other medical NLP tasks and investigating more efficient pre-training methods to reduce computational requirements.

References

[1] Z. Wen, X. H. Lu, and S. Reddy, "MeDAL: Medical abbreviation disambiguation dataset for natural language understanding pretraining," arXiv:2012.13978, 2020.

[2] Y. Wang, L. Wang, M. Rastegar-Mojarad, S. Moon, F. Shen, N. Afzal, S. Liu, Y. Zeng, S. Mehrabi, S. Sohn, and H. Liu, "Clinical information extraction applications: A literature review," Journal of Biomedical Informatics, vol. 77, pp. 34–49, 2018.

[3] W. Zhou, V. I. Torvik, and N. R. Smalheiser, "ADAM: another database of abbreviations in MEDLINE," Bioinformatics, vol. 22, no. 22, pp. 2813–2818, 2006.

[4] H. Xu, P. D. Stetson, and C. Friedman, "A study of abbreviations in clinical notes," AMIA Annual Symposium proceedings, pp. 821–825, 2007.

[5] R. Islamaj Dogan, G. C. Murray, A. Neveol, and Z. Lu, "Understanding PubMed(R) user search behavior through log analysis," Database, vol. 2009, 2009.

[6] S. Moon, S. Pakhomov, N. Liu, J. O. Ryan, and G. B. Melton, "A sense inventory for clinical abbreviations and acronyms created using clinical notes and medical dictionary resources," Journal of the American Medical Informatics Association, vol. 21, no. 2, pp. 299–307, 2014.

[7] I. Li, M. Yasunaga, M. Y. Nuzumlalı, C. Caraballo, S. Mahajan, H. Krumholz, and D. Radev, "A Neural Topic-Attention Model for Medical Term Abbreviation Disambiguation," arXiv:1910.14076, 2019.

[8] G. P. Finley, S. V. Pakhomov, R. McEwan, and G. B. Melton, "Towards Comprehensive Clinical Abbreviation Disambiguation Using Machine-Labeled Training Data," AMIA Annual Symposium proceedings, pp. 560–569, 2016.

[9] M. Skreta, A. Arbabi, J. Wang, and M. Brudno, "Training without training data: Improving the generalizability of automated medical abbreviation disambiguation," arXiv:1912.06174, 2019.

[10] Q. Jin, J. Liu, and X. Lu, "Deep Contextualized Biomedical Abbreviation Expansion," In BioNLP 2019, pp. 88–96, 2019.

[11] P. Grnarova, F. Schmidt, S. L. Hyland, and C. Eickhoff, "Neural Document Embeddings for Intensive Care Patient Mortality Prediction," arXiv:1612.00467, 2016.

[12] Y. Li, P. Nair, X. H. Lu, Z. Wen, Y. Wang, A. A. K. Dehaghi, Y. Miao, W. Liu, T. Ordog, J. M. Biernacka, E. Ryu, J. E. Olson, M. A. Frye, A. Liu, L. Guo, A. Marelli, Y. Ahuja, J. Davila-Velderrain, and M. Kellis, "Inferring multimodal latent topics from electronic health records," Nature communications, vol. 11, no. 1, pp. 2536, 2020.

[13] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching Word Vectors with Subword Information," Transactions of the Association for Computational Linguistics, vol. 5, pp. 135–146, 2017.
[14] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," arXiv:1409.0473, 2014.
[15] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators," In International Conference on Learning Representations, 2020.
[16] A. E. Johnson, T. J. Pollard, L. Shen, L. W. H. Lehman, M. Feng, M. Ghassemi, B. Moody, P. Szolovits, L. A. Celi, and R. G. Mark, "MIMIC-III, a freely accessible critical care database," Scientific Data, vol. 3, no. 1, pp. 1–9, 2016.
[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.
[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

APPENDIX: USER MANUAL

Environment Setup

1) **Clone the repository:**

```
git clone https://github.com/prashanthjaganathan/pretaining-language-models-for-medical-text.git

cd pretaining-language-models-for-medical-text
```

2) **(Optional) Create a virtual environment using conda:**

```
conda create --name venv python=3.10
```

3) **Install dependencies:**

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

Dataset Setup

MeDAL (Pretraining):

- Open `notebooks/medal_200_ctx_lstm_attention_demo.ipynb` and run the first two cells to download and load the dataset automatically.

MIMIC-IV Notes (Fine-tuning):

1) Download MIMIC-IV from the project's shared Google Drive link: `MIMIC-IV Dataset`
2) Move the `MIMIC-IV/` folder to the `dataset/` directory.
3) Unzip `discharge.csv.gz` inside `dataset/MIMIC-IV/`.

Medical Abstracts (Optional):

- Download the MATC dataset from the above link and place raw abstract files in `dataset/MATC/`.

Loading Trained Models

1) Download the full `trained_models/` folder from the shared link: `Trained models`
2) Replace `trained_models/` folder with the downloaded one.
3) Ensure the subfolders `models/`, `embeddings/`, and `tokenizers/` exist.

Running Experiments

Stage 1: Pretraining on MeDAL

- To run LSTM, check out: `notebooks/medal_lstm_demo.ipynb`
- To run LSTM + Attention (200 tokens), check out: `notebooks/medal_200_ctx_lstm_attention_demo.ipynb`
- To run Transformer, check out: `notebooks/transformers_final.ipynb`

Stage 2: Fine-tuning on MIMIC-IV

- To run LSTM, check out first part of `notebooks/mimic_finetuning_pretrained_lstm.ipynb`
- To run LSTM + Self Attention, check out second half of `notebooks/mimic_200_ctx_lstm_attention_demo.ipynb`

MATC Dataset (Optional):

- LSTM from scratch: `matc_lstm_demo.ipynb`
- LSTM fine-tuned: `matc_lstm_fine_tuned_demo.ipynb`
- LSTM + Attention: `notebooks/matc_lstm_attention_demo.ipynb`